

IMPLEMENTASI ALGORITMA *GREEDY* PADA *BOT NIRMALA* DALAM PERMAINAN *ROBOCODE TANK ROYALE*

Tugas Besar

Diajukan sebagai syarat menyelesaikan mata kuliah Strategi Algoritma Kelas RB di Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut Teknologi Sumatera



Oleh: Kelompok NIRMALA

Raisa Fazila Faisal (124140006)

Awra Hilmi (124140048)

Shafa Ayutyas Dewi (124140168)

Dosen Pengampu: Winda Yulita, M.Cs.

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA**

2026

DAFTAR PUSTAKA

BAB I	4
DESKRIPSI TUGAS	4
BAB II	9
2.1 Dasar Teori	9
2.2 Algoritma Greedy	9
2.2.1 Pengertian Algoritma Greedy	9
2.2.2 Karakteristik Algoritma Greedy	10
2.2.3 Kelebihan dan Kekurangan Algoritma Greedy	10
2.3 Implementasi Algoritma Greedy Pada Bot Robocode Tank Royale	11
2.4 Cara Kerja Program Bot	11
2.4.1 Inisialisasi Bot	11
2.4.2 Deteksi Musuh	11
2.4.3 Pengambilan Keputusan Menggunakan Greedy	12
2.4.4 Sistem Penyerangan	12
2.4.5 Sistem Pertahanan dan Pergerakan	12
BAB III	14
APLIKASI STRATEGI GREEDY	14
3.1 Penerapan Strategi Greedy pada Robocode Tank Royale	14
3.2 Mapping Persoalan Robocode Tank Royale ke Elemen Algoritma Greedy	14
3.2.1 Himpunan Kandidat (Candidate Set)	14
3.2.2 Himpunan Solusi (Solution Set)	14
3.2.3 Fungsi Seleksi (<i>Selection Function</i>)	15
3.2.4 Fungsi Kelayakan (<i>Feasibility Function</i>)	15
3.2.5 Fungsi Solusi (<i>Solution Function</i>)	15
3.2.6 Fungsi Objektif (<i>Objective Function</i>)	16
3.3 Eksplorasi Alternatif Strategi Greedy	16
3.3.1 Alternatif 1 - Greedy Berdasarkan Jarak Musuh	16
3.3.2 Alternatif 2 — Greedy Berdasarkan Energi Musuh	16
3.3.3 Alternatif 3 - Greedy Berdasarkan Tingkat Ancaman	17
3.3.4 Alternatif 4: Greedy Berdasarkan Efisiensi Energi	17
3.4 Analisis Efisiensi dan Efektivitas Strategi Greedy	18
3.5 Strategi Greedy yang Dipilih	18
BAB IV	20

IMPLEMENTASI DAN PENGUJIAN	20
4.1 Implementasi Strategi Greedy pada Bot Robocode Tank Royale	20
4.1.1 Implementasi BOTNIRMALA — Strategi Greedy Menyerang Musuh Terdekat	20
Konsep Strategi.....	20
4.1.2 Implementasi BOTKEDUA - Strategi Greedy Agresif	24
4.1.3 Implementasi BOTKETIGA - Strategi Bertahan Hidup	25
4.2 Struktur Data, Fungsi, dan Prosedur	28
4.2.1 Struktur Data yang Digunakan	28
4.2.2 Fungsi dan Prosedur Utama	28
4.3 Pengujian Program	29
4.3.1 Pengujian 1 - BOTNIRMALA vs BOTKEDUA.....	29
4.3.2 Pengujian 2 - BOTNIRMALA vs BOTKETIGA	29
4.3.3 Pengujian 3 - BOTNIRMALA vs BOTKEEMPAT	30
4.4 Analisis Hasil Pengujian.....	30
BAB V	32
KESIMPULAN DAN SARAN	32
5.1 Kesimpulan	32
5.2 Saran.....	32
LAMPIRAN	34
DAFTAR PUSTAKA	35

BAB I

DESKRIPSI TUGAS

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewati turn tersebut. Jika bot melewati turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol.

Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

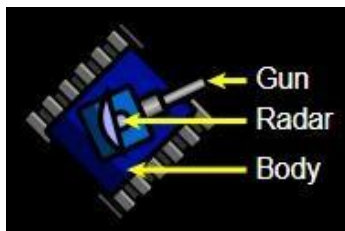
6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain.

Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.

Gun digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.

Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

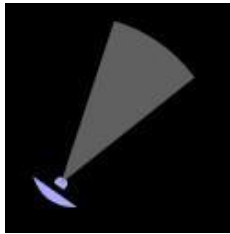
Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar damage** yang dibuat kepada bot musuh menggunakan peluru.

- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

BAB II

LANDASAN TEORI

2.1 Dasar Teori

Robocode Tank Royale merupakan permainan simulasi pertempuran tank berbasis pemrograman yang dikembangkan sebagai versi modern dari Robocode. Pada permainan ini, pemain tidak mengendalikan tank secara langsung, melainkan membuat program bot yang dapat bergerak, mencari musuh, menyerang, dan bertahan secara otomatis menggunakan algoritma tertentu. Setiap bot memiliki perilaku yang ditentukan melalui kode program sehingga hasil permainan bergantung pada strategi dan logika yang diterapkan.

Dalam Robocode Tank Royale, setiap bot dapat melakukan beberapa aksi utama seperti bergerak maju atau mundur, memutar badan tank, memutar radar, mendeteksi posisi musuh, menembakkan peluru, serta menghindari serangan lawan. Seluruh aksi tersebut dijalankan secara otomatis berdasarkan kondisi yang dibaca oleh bot selama permainan berlangsung.

Permainan ini sering digunakan sebagai media pembelajaran algoritma, kecerdasan buatan, dan pemrograman berorientasi objek karena memungkinkan pengembang untuk mengimplementasikan strategi tertentu ke dalam perilaku bot.

2.2 Algoritma Greedy

2.2.1 Pengertian Algoritma Greedy

Algoritma greedy merupakan metode penyelesaian masalah yang bekerja dengan memilih solusi terbaik pada setiap langkah secara lokal dengan harapan memperoleh solusi optimal secara keseluruhan. Kata “greedy” berarti “rakus”, yang menggambarkan cara algoritma ini dalam mengambil keputusan paling menguntungkan pada saat itu tanpa mempertimbangkan seluruh kemungkinan di masa depan.

Pendekatan greedy banyak digunakan karena memiliki proses yang sederhana, cepat, dan efisien dalam pengambilan keputusan. Pada setiap iterasi, algoritma akan memilih tindakan yang dianggap paling optimal berdasarkan kondisi saat ini.

Secara umum, langkah kerja algoritma greedy terdiri dari:

1. Menentukan himpunan kandidat solusi.
2. Memilih kandidat terbaik berdasarkan kondisi tertentu.
3. Memeriksa apakah pilihan tersebut memenuhi syarat solusi.
4. Menambahkan pilihan tersebut ke dalam solusi.
5. Mengulangi proses hingga tujuan tercapai.

Pada implementasi bot Robocode Tank Royale, algoritma greedy digunakan untuk menentukan tindakan terbaik yang harus dilakukan bot berdasarkan kondisi pertempuran secara real-time. Bot akan memilih aksi yang paling menguntungkan, misalnya:

1. Menembak ketika musuh berada dalam jangkauan.
2. Bergerak menghindar ketika musuh terlalu dekat.
3. Memutar radar untuk mencari target terdekat.
4. Mengejar musuh yang memiliki peluang serangan paling besar.

Dengan pendekatan ini, bot dapat bereaksi cepat terhadap situasi permainan tanpa perlu menghitung seluruh kemungkinan strategi yang kompleks.

2.2.2 Karakteristik Algoritma Greedy

Algoritma greedy memiliki beberapa karakteristik utama, yaitu:

1. **Local Optimum**
Pada setiap langkah, algoritma memilih keputusan terbaik berdasarkan kondisi saat ini.
2. **Tidak Melakukan Backtracking**
Keputusan yang telah diambil tidak akan diubah kembali.
3. **Proses Cepat dan Sederhana**
Karena hanya mempertimbangkan kondisi lokal, algoritma greedy memiliki kompleksitas yang relatif rendah.
4. **Efisien untuk Pengambilan Keputusan Real-Time**
Cocok digunakan pada permainan seperti Robocode Tank Royale yang membutuhkan respon cepat terhadap perubahan kondisi.

2.2.3 Kelebihan dan Kekurangan Algoritma Greedy

Kelebihan:

1. Proses pengambilan keputusan lebih cepat.
2. Implementasi algoritma lebih sederhana.
3. Cocok untuk sistem real-time.
4. Membutuhkan memori yang relatif kecil.

Kekurangan:

1. Tidak selalu menghasilkan solusi global terbaik.
2. Keputusan bersifat jangka pendek.
3. Strategi dapat diprediksi apabila aturan terlalu sederhana.

2.3 Implementasi Algoritma Greedy Pada Bot Robocode Tank Royale

Pada sistem bot yang dibuat, algoritma greedy diterapkan untuk menentukan aksi yang paling menguntungkan berdasarkan kondisi permainan saat itu. Bot akan terus membaca lingkungan permainan menggunakan radar, kemudian menentukan tindakan yang dianggap paling optimal.

Secara umum, alur kerja bot adalah sebagai berikut:

1. Bot memutar radar untuk mendeteksi keberadaan musuh.
2. Ketika musuh terdeteksi, bot membaca informasi seperti jarak, arah, dan posisi musuh.
3. Bot menentukan aksi terbaik berdasarkan kondisi tersebut.
4. Bot melakukan aksi seperti menembak, bergerak, atau menghindar.
5. Proses dilakukan terus-menerus selama pertandingan berlangsung.

Implementasi greedy terlihat pada proses pemilihan aksi secara langsung berdasarkan prioritas kondisi tertentu. Contohnya:

1. Jika musuh berada dekat, bot memilih menghindar agar tidak mudah terkena serangan.
2. Jika posisi musuh tepat berada di depan, bot memilih menembak.
3. Jika tidak ada musuh terdeteksi, bot memilih memutar radar untuk mencari target.
4. Jika energi bot rendah, bot dapat memilih bergerak defensif untuk menghindari serangan.

Dengan cara tersebut, bot selalu memilih tindakan yang dianggap paling menguntungkan pada saat itu.

2.4 Cara Kerja Program Bot

2.4.1 Inisialisasi Bot

Pada awal program dijalankan, bot akan melakukan inisialisasi terhadap komponen permainan seperti radar, arah gerakan, dan senjata. Selanjutnya bot akan masuk ke dalam loop utama untuk menjalankan aksi secara terus-menerus selama pertandingan berlangsung.

Tahap inisialisasi bertujuan agar bot memiliki kondisi awal yang siap digunakan untuk mendeteksi musuh dan melakukan pergerakan.

2.4.2 Deteksi Musuh

Bot menggunakan radar untuk mendeteksi keberadaan musuh di arena permainan. Radar akan diputar secara terus-menerus hingga terdapat lawan yang masuk ke area pemindaian.

Ketika radar mendeteksi musuh, sistem akan memperoleh beberapa informasi penting seperti:

1. Jarak musuh.
2. Sudut atau arah musuh.

3. Posisi relatif musuh.
4. Kecepatan pergerakan musuh.

Informasi tersebut digunakan sebagai dasar pengambilan keputusan.

2.4.3 Pengambilan Keputusan Menggunakan Greedy

Setelah data musuh diperoleh, bot akan menjalankan logika greedy untuk memilih aksi terbaik berdasarkan kondisi saat itu. Pada setiap kondisi, bot hanya memilih aksi yang dianggap paling menguntungkan saat itu tanpa menghitung seluruh kemungkinan strategi di masa depan.

2.4.4 Sistem Penyerangan

Bot akan menembakkan peluru ketika radar berhasil mengunci posisi musuh. Arah tembakan disesuaikan dengan posisi lawan agar peluang mengenai target lebih besar.

Dalam beberapa implementasi, kekuatan peluru dapat disesuaikan berdasarkan jarak musuh. Misalnya:

1. Musuh dekat → menggunakan kekuatan peluru besar.
2. Musuh jauh → menggunakan kekuatan peluru lebih kecil untuk menghemat energi.

Strategi tersebut juga termasuk bentuk penerapan greedy karena bot memilih penggunaan sumber daya yang paling efisien.

2.4.5 Sistem Pertahanan dan Pergerakan

Selain menyerang, bot juga melakukan pergerakan untuk menghindari serangan lawan. Bot dapat bergerak maju, mundur, atau berputar secara acak ketika mendeteksi ancaman.

Pergerakan dilakukan untuk:

1. Mengurangi kemungkinan terkena peluru.
2. Mengubah posisi agar sulit diprediksi.
3. Mendapatkan sudut serangan yang lebih baik.

Bot akan memilih gerakan yang paling aman berdasarkan kondisi saat itu.

2.5 Proses Menjalankan Bot pada Robocode Tank Royale

Secara umum, langkah menjalankan bot pada Robocode Tank Royale adalah sebagai berikut:

1. Menginstal Robocode Tank Royale dan bahasa pemrograman yang digunakan.
2. Membuat project bot menggunakan IDE seperti Visual Studio Code.
3. Menuliskan kode program bot.
4. Menjalankan server Robocode Tank Royale.
5. Menghubungkan bot ke server permainan.

6. Menjalankan pertandingan.
7. Mengamati perilaku bot selama permainan berlangsung.

Ketika pertandingan dimulai, bot akan bekerja secara otomatis berdasarkan algoritma yang telah diprogram. Semua keputusan seperti bergerak, menyerang, dan menghindar dilakukan tanpa campur tangan pengguna.

BAB III

APLIKASI STRATEGI GREEDY

3.1 Penerapan Strategi Greedy pada Robocode Tank Royale

Pada permainan Robocode Tank Royale, setiap bot harus mampu mengambil keputusan secara otomatis dan cepat berdasarkan kondisi arena permainan yang selalu berubah. Bot harus menentukan tindakan yang paling menguntungkan, seperti menyerang musuh, menghindari serangan, mengejar target, atau mempertahankan posisi.

Karena permainan berjalan secara real-time, maka diperlukan algoritma yang mampu memberikan keputusan dengan cepat dan efisien. Oleh sebab itu, algoritma greedy digunakan karena dapat memilih aksi terbaik berdasarkan kondisi saat ini tanpa harus menghitung seluruh kemungkinan strategi yang kompleks.

Dalam implementasinya, bot akan membaca kondisi arena menggunakan radar, kemudian menentukan aksi dengan prioritas tertinggi sesuai heuristic yang digunakan.

3.2 Mapping Persoalan Robocode Tank Royale ke Elemen Algoritma Greedy

Permasalahan dalam permainan Robocode Tank Royale dapat dipetakan ke dalam elemen-elemen algoritma greedy sebagai berikut.

3.2.1 Himpunan Kandidat (Candidate Set)

Himpunan kandidat merupakan kumpulan aksi yang dapat dipilih oleh bot selama permainan berlangsung.

Contoh himpunan kandidat pada bot Robocode Tank Royale:

1. Bergerak maju
2. Bergerak mundur
3. Berputar ke kanan atau kiri
4. Memutar radar
5. Menembak musuh
6. Mengejar musuh
7. Menghindari serangan
8. Bertahan pada posisi tertentu

Semua aksi tersebut menjadi pilihan yang dapat dipertimbangkan oleh bot dalam mengambil keputusan.

3.2.2 Himpunan Solusi (Solution Set)

Himpunan solusi merupakan kumpulan aksi yang dipilih oleh bot selama permainan berlangsung untuk mencapai tujuan permainan.

Contoh himpunan solusi:

1. Bot memutar radar.
2. Musuh terdeteksi.
3. Bot mendekati musuh.
4. Bot menembak target.
5. Bot menghindari serangan musuh.

Rangkaian tindakan tersebut membentuk strategi bot selama pertandingan berlangsung.

3.2.3 Fungsi Seleksi (*Selection Function*)

Fungsi seleksi digunakan untuk memilih aksi yang dianggap paling menguntungkan berdasarkan kondisi saat ini.

Contoh fungsi seleksi pada bot:

1. Memilih musuh yang paling dekat
2. Memilih posisi paling aman
3. Memilih waktu terbaik untuk menembak
4. Memilih arah gerakan dengan risiko terkena peluru paling kecil

Bot akan memilih aksi dengan nilai prioritas tertinggi berdasarkan heuristic yang digunakan.

3.2.4 Fungsi Kelayakan (*Feasibility Function*)

Fungsi kelayakan digunakan untuk menentukan apakah suatu aksi dapat dilakukan atau tidak.

Contoh:

1. Bot hanya dapat menembak jika energi mencukupi.
2. Bot hanya dapat bergerak jika tidak menabrak dinding arena.
3. Bot mengejar musuh hanya jika posisi masih aman.
4. Bot menghindar ketika musuh terlalu dekat.

Fungsi ini memastikan tindakan yang dipilih tetap valid dan dapat dijalankan.

3.2.5 Fungsi Solusi (*Solution Function*)

Fungsi solusi digunakan untuk menentukan apakah tindakan yang dilakukan telah mendukung tujuan permainan.

Pada Robocode Tank Royale, tujuan utama bot adalah:

1. Bertahan hidup selama mungkin
2. Mengurangi energi musuh
3. Mengalahkan lawan
4. Menjadi pemenang pertandingan

Jika tindakan yang dipilih mampu meningkatkan peluang kemenangan, maka solusi dianggap baik.

3.2.6 Fungsi Objektif (*Objective Function*)

Fungsi objektif digunakan untuk mengukur kualitas keputusan yang diambil oleh bot.

Beberapa parameter yang dapat digunakan sebagai fungsi objektif antara lain:

1. Memaksimalkan jumlah tembakan yang mengenai musuh
2. Meminimalkan kerusakan yang diterima
3. Memaksimalkan sisa energi bot
4. Memaksimalkan peluang kemenangan
5. Meminimalkan peluru yang meleset

Bot akan memilih aksi yang memberikan nilai objektif terbaik.

3.3 Eksplorasi Alternatif Strategi Greedy

Dalam pengembangan bot Robocode Tank Royale, terdapat beberapa alternatif strategi greedy yang dapat digunakan. Setiap strategi memiliki heuristic yang berbeda dalam menentukan keputusan terbaik.

3.3.1 Alternatif 1 - Greedy Berdasarkan Jarak Musuh

Strategi ini memilih musuh yang memiliki jarak paling dekat dengan bot.

Cara Kerja

1. Radar mendeteksi seluruh musuh.
2. Bot menghitung jarak setiap musuh.
3. Bot memilih musuh terdekat.
4. Bot bergerak dan menyerang target tersebut.

Kelebihan

1. Mudah diimplementasikan
2. Respon cepat
3. Peluang tembakan mengenai target lebih besar

Kekurangan

1. Bot lebih mudah terkena serangan
2. Tidak mempertimbangkan kondisi energi dan posisi lawan

3.3.2 Alternatif 2 — Greedy Berdasarkan Energi Musuh

Strategi ini memilih musuh dengan energi paling rendah agar lebih mudah dikalahkan.

Heuristic

Pilih target dengan energi minimum.

Cara Kerja

1. Bot mendeteksi seluruh musuh.
2. Bot membandingkan energi lawan.

3. Bot memilih musuh dengan energi terendah.
4. Bot fokus menyerang target tersebut.

Kelebihan

1. Memungkinkan eliminasi musuh lebih cepat
2. Mengurangi jumlah lawan di arena

Kekurangan

1. Musuh dengan energi rendah belum tentu paling dekat
2. Bot dapat bergerak ke posisi berbahaya saat mengejar target

3.3.3 Alternatif 3 - Greedy Berdasarkan Tingkat Ancaman

Strategi ini memilih tindakan berdasarkan tingkat ancaman musuh terhadap bot.

Heuristic

Pilih musuh dengan tingkat ancaman tertinggi.

Tingkat ancaman dapat ditentukan berdasarkan:

1. Jarak musuh
2. Arah tembakan musuh
3. Kecepatan musuh
4. Posisi musuh terhadap bot

Cara Kerja

1. Bot mendeteksi musuh.
2. Bot menghitung tingkat ancaman setiap lawan.
3. Bot memilih lawan paling berbahaya.
4. Bot menyerang atau menghindar sesuai kondisi.

Kelebihan

1. Lebih defensif
2. Mengurangi risiko terkena serangan
3. Meningkatkan peluang bertahan hidup

Kekurangan

1. Perhitungan lebih kompleks
2. Membutuhkan lebih banyak parameter

3.3.4 Alternatif 4: Greedy Berdasarkan Efisiensi Energi

Strategi ini memilih aksi yang paling hemat energi.

Heuristic

Pilih aksi dengan penggunaan energi paling efisien.

Cara Kerja

1. Bot memantau sisa energi.
2. Bot menyesuaikan kekuatan tembakan berdasarkan jarak musuh.
3. Bot mengurangi pergerakan yang tidak diperlukan.
4. Bot bermain defensif ketika energi rendah.

Kelebihan

1. Menghemat energi
2. Cocok untuk pertandingan jangka panjang
3. Mengurangi resiko kehabisan energi

Kekurangan

1. Serangan menjadi kurang agresif
2. Eliminasi musuh lebih lambat

3.4 Analisis Efisiensi dan Efektivitas Strategi Greedy

Analisis efisiensi dan efektivitas dari setiap alternatif strategi greedy:

1. Strategi berdasarkan jarak musuh memiliki proses sederhana sehingga sangat cepat dijalankan, namun bot menjadi lebih rentan terkena serangan.
2. Strategi berdasarkan energi musuh efektif untuk mengurangi jumlah lawan dengan cepat, tetapi dapat membawa bot ke area berbahaya.
3. Strategi berdasarkan tingkat ancaman memiliki efektivitas paling baik karena mempertimbangkan keselamatan dan peluang bertahan hidup.
4. Strategi berdasarkan efisiensi energi sangat baik dalam menjaga energi bot, tetapi kurang agresif dalam menyerang lawan.

3.5 Strategi Greedy yang Dipilih

Berdasarkan hasil analisis, strategi greedy yang dipilih untuk diimplementasikan pada program adalah Strategi Greedy Berdasarkan Tingkat Ancaman. Strategi ini dipilih karena memiliki keseimbangan terbaik antara kemampuan menyerang dan bertahan.

Alasan Pemilihan Strategi

1. Meningkatkan peluang bertahan hidup.
2. Bot lebih adaptif terhadap kondisi arena.
3. Mengurangi risiko terkena serangan.
4. Memberikan keputusan yang lebih stabil.
5. Cocok digunakan pada permainan real-time seperti Robocode Tank Royale.

Pertimbangan Pemilihan

Strategi berdasarkan tingkat ancaman dipilih karena tidak hanya fokus menyerang, tetapi juga mempertimbangkan keamanan bot. Dengan mempertimbangkan jarak, arah, dan posisi musuh, bot dapat menentukan tindakan yang lebih optimal.

Strategi ini memungkinkan bot untuk:

1. Menghindari musuh yang terlalu berbahaya
2. Menyerang ketika peluang keberhasilan tinggi
3. Bermanuver secara lebih aman
4. Menyesuaikan tindakan berdasarkan kondisi arena

Selain itu, strategi ini lebih fleksibel dibandingkan strategi yang hanya berfokus pada jarak atau energi musuh.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Strategi Greedy pada Bot Robocode Tank Royale

Pada tahap implementasi, strategi greedy diterapkan ke dalam bot Robocode Tank Royale menggunakan bahasa pemrograman C#. Implementasi dilakukan dengan membuat empat bot berbeda yang masing-masing memiliki heuristic tersendiri dalam menentukan tindakan selama pertandingan berlangsung.

Setiap bot dibangun menggunakan konsep *event-driven programming* yang disediakan oleh Robocode Tank Royale. Bot akan merespon kondisi tertentu melalui event seperti:

1. `OnScannedBot()` ketika musuh terdeteksi
2. `OnHitWall()` ketika bot menabrak dinding
3. `OnHitByBullet()` ketika bot terkena peluru

Strategi greedy diterapkan pada proses pengambilan keputusan di setiap event tersebut.

4.1.1 Implementasi BOTNIRMALA — Strategi Greedy Menyerang Musuh Terdekat

BOTNIRMALA merupakan bot utama yang menggunakan strategi greedy menyerang musuh secara langsung setelah radar mendeteksi lawan. Bot ini memiliki perilaku agresif dengan fokus memberikan damage secepat mungkin.

Konsep Strategi

Strategi greedy yang digunakan pada BOTNIRMALA adalah memilih tindakan menyerang secara langsung ketika terdapat target yang terdeteksi radar.

Heuristic yang digunakan:

1. Jika musuh ditemukan → langsung tembak
2. Jika posisi masih aman → dekati musuh
3. Jika menabrak dinding → ubah arah

Strategi ini dipilih karena memberikan respon cepat terhadap kondisi arena.

Alur Kerja BOTNIRMALA

1. Bot memulai permainan.
2. Bot melakukan patrol sederhana.
3. Radar diputar 360° untuk mencari musuh.
4. Ketika musuh ditemukan:
 - a. bot langsung menembak
 - b. bot bergerak mendekati target
5. Jika menabrak dinding:
 - a. bot mundur
 - b. bot mengubah arah
6. Proses dilakukan terus menerus selama pertandingan berlangsung.

Pseudocode BOTNIRMALA

```
START

Inisialisasi bot
Atur warna body, radar, turret, dan peluru

WHILE pertandingan masih berjalan DO

    Bergerak maju sejauh 150

    Putar radar dan senjata 360 derajat
    // mencari musuh di arena

    Belok kanan 45 derajat
    // mengubah arah gerakan agar tidak mudah ditebak

    Bergerak mundur 100
    // menghindari posisi diam terlalu lama

END WHILE

EVENT OnScannedBot
    // dijalankan ketika radar mendeteksi musuh

    Tembak musuh dengan power 3
    // serangan agresif untuk damage besar

    Bergerak maju 50
    // mendekati musuh setelah menyerang
END EVENT

EVENT OnHitWall
    // dijalankan ketika bot menabrak dinding

    Bergerak mundur 100

    Belok kanan 90 derajat
    // mencari arah baru
END EVENT

END
```

Penjelasan Implementasi Program BOTNIRMALA

Fungsi Main()

```
static void Main(string[] args)
{
    new BOTNIRMALA().Start();
}
```

Fungsi Main() digunakan sebagai titik awal program. Fungsi ini akan membuat objek BOTNIRMALA lalu menjalankan bot menggunakan method Start().

Constructor BOTNIRMALA

```
BOTNIRMALA() : base(BotInfo.FromFile("BOTNIRMALA.json")) { }
```

Constructor digunakan untuk membaca file konfigurasi bot berupa BOTNIRMALA.json yang berisi identitas bot.

Fungsi Run()

```
public override void Run()
```

Fungsi Run() merupakan fungsi utama yang berjalan selama pertandingan berlangsung.

Pada fungsi ini bot melakukan:

1. Pengaturan warna
2. Pergerakan utama
3. Rotasi radar
4. Perubahan arah

Bot menjalankan loop:

```
while (IsRunning)
```

Loop tersebut memastikan bot terus aktif selama pertandingan berlangsung.

Event OnScannedBot()

```
public override void OnScannedBot(ScannedBotEvent e)
```

Event ini dijalankan ketika radar mendeteksi musuh.

Bot akan:

```
Fire(3);
```

Menembak dengan power 3 untuk memberikan damage besar.

Kemudian:

```
Forward(50);
```

Bot mendekati musuh untuk mempertahankan tekanan serangan.

Event OnHitWall()

```
public override void OnHitWall(HitWallEvent e)
```

Event ini dijalankan ketika bot menabrak dinding.

Bot akan:

1. mundur
2. mengubah arah
3. melanjutkan pergerakan

Strategi ini mencegah bot terjebak di sudut arena.

Source Code BOTNIRMALA

```
using System;
using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;

public class BOTNIRMALA : Bot
{
    static void Main(string[] args)
    {
        new BOTNIRMALA().Start();
    }

    BOTNIRMALA() : base(BotInfo.FromFile("BOTNIRMALA.json")) { }

    public override void Run()
    {
        // Tema Biru
        BodyColor = Color.FromArgb(0x1E, 0x90, 0xFF);
        TurretColor = Color.FromArgb(0x00, 0xBF, 0xFF);
        RadarColor = Color.FromArgb(0x87, 0xCE, 0xFA);
        BulletColor = Color.FromArgb(0xFF, 0x45, 0x00);
        ScanColor = Color.FromArgb(0xFF, 0xFF, 0x00);
        TracksColor = Color.FromArgb(0x19, 0x19, 0x70);
        GunColor = Color.FromArgb(0x00, 0x00, 0x8B);

        while (IsRunning)
        {
            Forward(150);

            TurnGunRight(360);

            TurnRight(45);

            Back(100);
        }
    }

    // Heuristic: menyerang musuh terdekat
    public override void OnScannedBot(ScannedBotEvent e)
    {
        Fire(3);

        Forward(50);
    }

    public override void OnHitWall(HitWallEvent e)
    {

```

```
        Back(100);

        TurnRight(90);
    }
}
```

4.1.2 Implementasi BOTKEDUA - Strategi Greedy Agresif

BOTKEDUA menggunakan strategi menyerang secara terus menerus tanpa mempertimbangkan banyak aspek pertahanan.

Konsep Strategi

Heuristic yang digunakan:

1. Jika menemukan musuh → langsung menyerang
2. Bot terus bergerak mendekati target
3. Mengutamakan tekanan serangan dibanding pertahanan

Strategi ini membuat bot lebih agresif namun memiliki risiko tinggi.

Pseudocode BOTKEDUA

```
START

Atur warna bot

WHILE pertandingan masih berjalan DO

    Bergerak maju 200

    Putar radar 360 derajat

    Belok kiri 90 derajat

    Bergerak maju 100

END WHILE

EVENT OnScannedBot

    Tembak musuh power 2

    Belok kanan 30 derajat

    Bergerak maju 80
```

```
END EVENT
```

```
EVENT OnHitWall
```

```
    Belok kanan 180 derajat
```

```
    Bergerak maju 100
```

```
END EVENT
```

```
END
```

Analisis BOTKEDUA

Kelebihan

1. Serangan cepat
2. Tekanan tinggi terhadap lawan
3. Mobilitas cukup baik

Kekurangan

1. Kurang mempertimbangkan posisi aman
2. Mudah terkena serangan
3. Konsumsi energi lebih besar

4.1.3 Implementasi BOTKETIGA - Strategi Bertahan Hidup

BOTKETIGA menerapkan strategi defensif dengan fokus menghindari serangan lawan.

Konsep Strategi

Heuristic:

1. Setelah menyerang → langsung menghindar
2. Jika terkena peluru → pindah posisi
3. Mengurangi kemungkinan terkena damage

Strategi ini lebih fokus pada survival dibanding serangan.

Pseudocode BOTKETIGA

```
START
```

```
Atur warna bot
```

```
WHILE pertandingan masih berjalan DO
```

```
    Bergerak maju 100
```

```
    Belok kanan 120 derajat
```

```
    Putar radar 360 derajat

    Bergerak mundur 50

END WHILE

EVENT OnScannedBot

    Tembak musuh power 1

    Bergerak mundur 120

    Belok kiri 90 derajat

END EVENT

EVENT OnHitByBullet

    Belok kanan 90 derajat

    Bergerak maju 150

END EVENT

END
```

Analisis BOTKETIGA

Kelebihan

1. Sulit terkena peluru
2. Pergerakan lebih dinamis
3. Bertahan lebih lama

Kekurangan

1. Damage kecil
2. Kurang agresif
3. Pertandingan cenderung lebih lama

4.1.4 Implementasi BOTKEEMPAT - Strategi Patrol Arena

BOTKEEMPAT menerapkan strategi patroli arena sambil melakukan scanning terhadap musuh.

Konsep Strategi

Heuristic:

1. Bergerak mengelilingi arena
2. Scan musuh secara terus menerus
3. Menyerang ketika target ditemukan

Strategi ini fokus pada eksplorasi arena.

Pseudocode BOTKEEMPAT

```
START

Atur warna bot

WHILE pertandingan masih berjalan DO

    Bergerak maju 250

    Belok kanan 90 derajat

    Putar radar 360 derajat

END WHILE

EVENT OnScannedBot

    Tembak musuh power 2

END EVENT

EVENT OnHitWall

    Bergerak mundur 100

    Belok kiri 90 derajat

END EVENT

END
```

Analisis BOTKEEMPAT

Kelebihan

1. Mudah menemukan musuh
2. Pergerakan stabil
3. Cocok untuk eksplorasi arena

Kekurangan

1. Kurang efektif duel jarak dekat
2. Serangan tidak terlalu agresif

4.2 Struktur Data, Fungsi, dan Prosedur

4.2.1 Struktur Data yang Digunakan

Bot memanfaatkan beberapa struktur data bawaan Robocode Tank Royale.

Struktur Data	Kegunaan
BotInfo	Menyimpan konfigurasi dan identitas bot
ScannedBotEvent	Menyimpan data musuh yang terdeteksi
HitWallEvent	Menyimpan data tabrakan dinding
HitByBulletEvent	Menyimpan data ketika terkena peluru
Color	Mengatur warna bot

4.2.2 Fungsi dan Prosedur Utama

Fungsi Run()

Merupakan fungsi utama yang dijalankan selama pertandingan berlangsung.

Fungsi ini mengatur:

1. Pergerakan bot
2. Rotasi radar
3. Patrol arena
4. Pergantian arah

Fungsi OnScannedBot()

Digunakan ketika radar mendeteksi lawan.

Fungsi ini bertugas:

1. Menyerang musuh
2. Menentukan respon terhadap lawan
3. Menjalankan heuristic greedy

Fungsi OnHitWall()

Digunakan ketika bot menabrak dinding.

Fungsi ini mencegah bot:

1. terjebak di sudut
2. berhenti bergerak
3. kehilangan mobilitas

Fungsi OnHitByBullet()

Digunakan ketika bot terkena peluru.

Fungsi ini membantu bot:

1. menghindari serangan lanjutan
2. berpindah posisi
3. memperbesar peluang bertahan hidup

4.3 Pengujian Program

Pengujian dilakukan menggunakan format pertandingan 1 vs 1.

Bot utama:

- BOTNIRMALA

Bot lawan:

- BOTKEDUA
- BOTKETIGA
- BOTKEEMPAT

Tujuan pengujian:

1. Mengukur efektivitas strategi greedy
2. Mengamati perilaku bot
3. Membandingkan heuristic tiap strategi

4.3.1 Pengujian 1 - BOTNIRMALA vs BOTKEDUA

Skenario Pengujian

BOTNIRMALA melawan bot agresif yang terus menyerang.

Hasil Pengujian

Bot	Hasil
BOTNIRMALA	Menang
BOTKEDUA	Kalah

Analisis Pengujian

BOTKEDUA memiliki pola serangan agresif sehingga sering bergerak mendekati lawan.

BOTNIRMALA berhasil memanfaatkan kondisi tersebut karena:

1. Radar aktif terus menerus
2. Serangan dilakukan lebih cepat
3. Posisi lawan mudah diprediksi

Namun BOTNIRMALA juga menerima damage cukup besar karena terlalu sering mendekati musuh.

4.3.2 Pengujian 2 - BOTNIRMALA vs BOTKETIGA

Skenario Pengujian

BOTNIRMALA melawan bot defensif yang lebih fokus menghindar.

Hasil Pengujian

Bot	Hasil
BOTNIRMALA	Menang
BOTKETIGA	Kalah

Analisis Pengujian

BOTKETIGA memiliki pergerakan sulit diprediksi karena sering menghindar setelah menyerang.

Strategi BOTNIRMALA tetap berhasil karena:

1. Agresivitas serangan lebih tinggi
2. Tekanan terhadap lawan terus dilakukan
3. BOTKETIGA terlalu defensif

Pertandingan berlangsung lebih lama dibanding pengujian pertama.

4.3.3 Pengujian 3 - BOTNIRMALA vs BOTKEEMPAT

Skenario Pengujian

BOTNIRMALA melawan bot patroli arena.

Hasil Pengujian

Bot	Hasil
BOTNIRMALA	Menang
BOTKEEMPAT	Kalah

Analisis Pengujian

BOTKEEMPAT memiliki mobilitas cukup stabil karena melakukan patrol arena.

Namun strategi patrol memiliki kelemahan:

1. kurang fokus menyerang
2. respon terhadap lawan lebih lambat
3. kurang efektif duel jarak dekat

BOTNIRMALA berhasil menang karena lebih cepat memberikan damage.

4.4 Analisis Hasil Pengujian

Berdasarkan seluruh pengujian yang dilakukan, strategi greedy yang diterapkan pada BOTNIRMALA cukup efektif untuk pertandingan 1 vs 1.

Keberhasilan Strategi Greedy

Strategi greedy berhasil memberikan:

1. Respon cepat
2. Serangan agresif
3. Pengambilan keputusan sederhana
4. Tekanan besar terhadap lawan

BOTNIRMALA mampu memenangkan seluruh pertandingan karena heuristic menyerang langsung cukup efektif.

Kelemahan Strategi Greedy

Meskipun efektif, strategi greedy tidak selalu menghasilkan solusi optimal.

Beberapa kelemahan yang ditemukan:

1. Bot terlalu fokus menyerang.
2. Bot kurang mempertimbangkan posisi aman.
3. Bot belum mampu memprediksi arah musuh.
4. Strategi hanya berdasarkan kondisi saat ini.

Ketika menghadapi lawan yang sangat cepat atau memiliki strategi prediksi yang lebih baik, strategi greedy sederhana dapat mengalami penurunan performa.

Kondisi Strategi Tidak Optimal

Strategi greedy menjadi kurang optimal ketika:

- Musuh bergerak terlalu cepat
- Radar kehilangan target
- Bot terjebak di sudut arena
- Energi bot mulai rendah

Dalam kondisi tersebut, bot tetap memilih aksi lokal terbaik meskipun belum tentu menjadi keputusan global terbaik.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa algoritma greedy berhasil diterapkan pada permainan Robocode Tank Royale melalui pembuatan empat bot dengan heuristic yang berbeda. Setiap bot memiliki strategi pengambilan keputusan yang berfokus pada pemilihan aksi terbaik berdasarkan kondisi saat itu (*local optimum*).

Empat strategi greedy yang diterapkan meliputi:

1. Strategi menyerang musuh terdekat pada BOTNIRMALA
2. Strategi agresif pada BOTKEDUA
3. Strategi bertahan hidup pada BOTKETIGA
4. Strategi patrol arena pada BOTKEEMPAT

Dari seluruh strategi yang diuji, BOTNIRMALA dipilih sebagai bot utama karena memiliki performa paling efektif pada pertandingan 1 vs 1. BOTNIRMALA mampu memberikan respon cepat terhadap musuh dengan cara langsung menyerang setelah radar mendeteksi target. Strategi ini menghasilkan tekanan serangan yang tinggi sehingga mampu memenangkan pertandingan melawan bot lainnya.

Hasil pengujian menunjukkan bahwa strategi greedy memiliki beberapa kelebihan, yaitu:

1. Proses pengambilan keputusan cepat,
2. Implementasi sederhana,
3. Cocok digunakan pada sistem real-time,
4. Efektif untuk pertandingan dengan respon cepat.

Namun strategi greedy juga memiliki keterbatasan karena hanya mempertimbangkan kondisi saat ini tanpa memperhitungkan kemungkinan jangka panjang. Pada kondisi tertentu, seperti ketika menghadapi musuh yang bergerak sangat cepat atau memiliki pola pergerakan tidak terduga, strategi greedy dapat menjadi kurang optimal.

Secara keseluruhan, penerapan algoritma greedy pada Robocode Tank Royale berhasil dilakukan dan mampu menghasilkan bot yang dapat bertanding secara efektif di arena permainan.

5.2 Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa saran untuk pengembangan penelitian selanjutnya, yaitu:

1. Menambahkan sistem prediksi pergerakan musuh agar bot dapat menembak dengan akurasi lebih tinggi.
2. Mengembangkan strategi hybrid dengan menggabungkan algoritma greedy dan algoritma lain seperti minimax atau fuzzy logic.

3. Menambahkan sistem manajemen energi sehingga bot dapat menentukan kapan harus menyerang dan kapan harus bertahan.
4. Mengembangkan sistem navigasi arena yang lebih kompleks agar bot tidak mudah terjebak di sudut arena.
5. Mengimplementasikan machine learning atau reinforcement learning agar bot dapat belajar dari pertandingan sebelumnya.
6. Melakukan pengujian dengan lebih banyak variasi bot dan skenario pertandingan agar hasil analisis menjadi lebih akurat.

LAMPIRAN

A. Repository Github

https://github.com/awaa02a/Tubes_Nirmala.git

DAFTAR PUSTAKA

- Introduction to Algorithms Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press, 2009.
- Algorithm Design Kleinberg, J., & Tardos, É. *Algorithm Design*. Pearson Education, 2006.
- [Robocode Tank Royale Documentation](#)
- Microsoft Visual Studio Microsoft Corporation. *Visual Studio Documentation*. Microsoft Docs.
- C Sharp Microsoft Corporation. *C# Programming Guide*. Microsoft Docs.
- Artificial Intelligence Russell, S., & Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson, 2010.
- Greedy Algorithm Munir, R. *Matematika Diskrit*. Informatika, Bandung, 2018